

Your Code

```
1 names = [  
2 " Arun Kumar ",  
3 "ARUN KUMAR",  
4 " arun kumar",  
5 "Bala Krishnan",  
6 "BALA KRISHNAN ",  
7 "Charan",  
8 " charan ",  
9 "DIVYA DEVI",  
10 "Divya Devi"  
11 ]  
12  
13 normalized = [name.strip().lower() for name in names]  
14 print("Normalized:")  
15 print(normalized)  
16  
17 unique_customers = []  
18 for name in normalized:  
19     if name not in unique_customers:  
20         unique_customers.append(name)  
21 unique_customers.sort()  
22  
23 print("\nUnique Customers:")
```

Input

stdin input

Output

```
[ 'arun kumar', 'bala krishnan',  
'charan', 'divya devi']
```

```
Most Frequent Customer:  
arun kumar
```

```
1 products = [  
2     ("Laptop", 5, 55000),  
3     ("Mouse", 25, 700),  
4     ("Keyboard", 8, 1200),  
5     ("Monitor", 12, 15000),  
6     ("Printer", 4, 12000),  
7     ("Headset", 15, 2500)  
8 ]  
9  
10 print("Inventory Values:")  
11 inventory_values = []  
12 for name, qty, price in products:  
13     value = qty * price  
14     inventory_values.append((name, value))  
15     print(f"{name} : {value}")  
16  
17 total_value = sum(v for _, v in inventory_values)  
18 print(f"\nTotal Inventory Value: {total_value}")  
19  
20 print("\nLow Stock:")  
21 for name, qty, price in products:  
22     if qty < 10:  
23         print(name)  
24
```

stdin input

Output

```
Inventory Values:  
Laptop : 275000  
Mouse : 17500  
Keyboard : 9600  
Monitor : 180000  
Printer : 48000
```

```
1 import string
2
3 feedback = """
4 Python programming is easy to learn.
5 Python programming is powerful.
6 Learning Python requires practice.
7 Practice makes programming skills better.
8 """
9
10 text = feedback.lower()
11 text = text.translate(str.maketrans('', '', string.punctuation))
12 words = text.split()
13
14 total_words = len(words)
15 unique_words = set(words)
16 num_unique = len(unique_words)
17
18 freq_python = words.count("python")
19 freq_programming = words.count("programming")
20
21 repeated_words = [word for word in unique_words if words.count(word) >
22
23 longest_word = max(unique_words, key=len)
24
```

stdin input

Output

```
Total Words: 19
Unique Words: 13

Frequency of python: 3
Frequency of programming: 3
```

Your Code

```
1 employees = [  
2     ("E101", "Arun", ["Python", "SQL", "ML"]),  
3     ("E102", "Bala", ["Java", "SQL", "HTML"]),  
4     ("E103", "Charan", ["Python", "ML", "SQL"]),  
5     ("E104", "Divya", ["Python", "Java", "Cloud"]),  
6     ("E105", "Esha", ["Python", "ML", "Cloud"])  
7 ]  
8  
9 python_emp = [name for _, name, skills in employees if "Python" in skills]  
10 print("Python:")  
11 print(", ".join(python_emp))  
12 print()  
13  
14 python_ml = [name for _, name, skills in employees if "Python" in skills and "ML" in skills]  
15 print("Python + ML:")  
16 print(", ".join(python_ml))  
17 print()  
18  
19 sql_not_java = [name for _, name, skills in employees if "SQL" in skills and "Java" not in skills]  
20 print("SQL but not Java:")  
21 print(", ".join(sql_not_java))  
22 print()
```

Input

stdin input

Output

```
Python:  
Arun, Charan, Divya, Esha  
  
Python + ML:  
Arun, Charan, Esha
```

Your Code

```
1 transactions = [  
2     ("T101", "Deposit", 5000),  
3     ("T102", "Withdraw", 2000),  
4     ("T103", "Deposit", 8000),  
5     ("T104", "Withdraw", 1500),  
6     ("T105", "Deposit", 3000),  
7     ("T106", "Withdraw", 7000)  
8 ]  
9  
10 total_deposit = sum(a for _, t, a in transactions if t == "Deposit")  
11 total_withdraw = sum(a for _, t, a in transactions if t == "Withdraw")  
12 net_balance = total_deposit - total_withdraw  
13  
14 largest_deposit = max(a for _, t, a in transactions if t == "Deposit")  
15 largest_withdraw = max(a for _, t, a in transactions if t == "Withdraw")  
16  
17 greater_than_4000 = [tid for tid, _, a in transactions if a > 4000]  
18  
19 deposit_count = sum(1 for _, t, _ in transactions if t == "Deposit")  
20 withdraw_count = sum(1 for _, t, _ in transactions if t == "Withdraw")  
21  
22 avg_transaction = sum(a for _, _, a in transactions) / len(transactions)
```

Input

stdin input

Output

```
Total Deposit: 16000  
Total Withdrawal: 10500  
Net Balance: 5500  
  
Largest Deposit: 8000  
Largest Withdrawal: 7000
```

Your Code

```
1 passwords = [  
2     "Python@123",  
3     "python123",  
4     "PYTHON@123",  
5     "Py@45",  
6     "Python123",  
7     "Java#4567",  
8     "Admin@2026"  
9 ]  
10  
11 valid_passwords = []  
12 invalid_passwords = []  
13  
14 for p in passwords:  
15     errors = []  
16     if len(p) < 8:  
17         errors.append("Less than 8 characters")  
18     if not any(c.isupper() for c in p):  
19         errors.append("No uppercase")  
20     if not any(c.islower() for c in p):  
21         errors.append("No lowercase")  
22     if not any(c.isdigit() for c in p):  
23         errors.append("No digit")
```

Input

stdin input

Output

```
Valid:  
Python@123  
Java#4567  
Admin@2026  
  
Invalid:
```

9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

```
['arun', 'bala', 'charan']
```

Your Code

```
1 records = [  
2     (" Arun ", "Python"),  
3     ("ARUN", "Python"),  
4     (" Bala ", "Java"),  
5     ("bala", "Java"),  
6     ("Charan", "Python")  
7 ]  
8  
9 names = []  
10  
11 for record in records:  
12     name = record[0].strip().lower()  
13     if name not in names:  
14         names.append(name)  
15  
16 print(names)
```

Input

stdin input

Output

```
['arun', 'bala', 'charan']
```

Your Code

```
1 from collections import defaultdict
2
3 students = [
4     (" A001 ", "ARUN KUMAR", [78, 85, 92]),
5     ("A002", "Bala", [45, 67, 52]),
6     (" a003", "arun kumar", [88, 91, 95]),
7     ("A004", "CHARAN", [72, 76, 81]),
8     ("A005", "Divya ", [55, 48, 62])
9 ]
10
11 normalized_records = []
12 totals = defaultdict(int)
13 averages = {}
14 passed_all = []
15 failed_at_least_one = []
16 name_counts = defaultdict(int)
17
18 for sid, name, marks in students:
19     sid_n = sid.strip().upper()
20     name_n = name.strip().lower()
21     normalized_records.append((sid_n, name_n, marks))
```

Input

stdin input

Output

```
Normalized Records:
A001 arun kumar [78, 85, 92]
A002 bala [45, 67, 52]
A003 arun kumar [88, 91, 95]
A004 charan [72, 76, 81]
A005 divya [55, 48, 62]
```


Your Code

```
1 students = [  
2     ("Arun", [78, 85, 92, 67]),  
3     ("Bala", [45, 67, 55, 49]),  
4     ("Charan", [90, 92, 88, 95]),  
5     ("Divya", [65, 72, 70, 68]),  
6     ("Esha", [88, 76, 91, 84])  
7 ]  
8  
9 totals = {}  
10 averages = {}  
11  
12 print("Student Totals:")  
13 for name, marks in students:  
14     total = sum(marks)  
15     totals[name] = total  
16     print(f"{name} : {total}")  
17  
18 print("\nStudent Averages:")  
19 for name, marks in students:  
20     avg = sum(marks) / len(marks)  
21     averages[name] = avg
```

Input

stdin input

Output

```
Student Totals:  
Arun : 322  
Bala : 216  
Charan : 365  
Divya : 275  
Esha : 339
```